

Programmazione 1

Corso c

24/10/2023
> Gli array

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

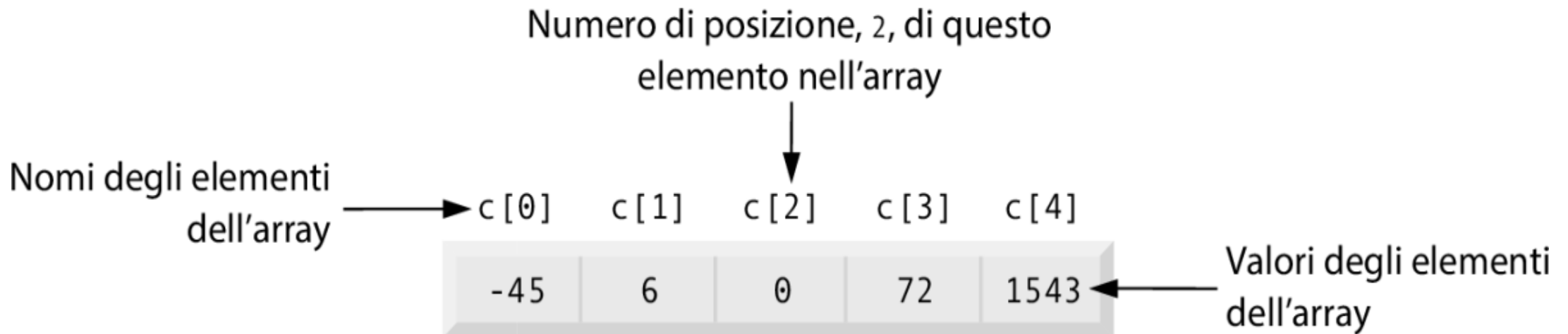
- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Gli Array

- Un array è un gruppo di elementi dello stesso tipo immagazzinati in modo contiguo nella memoria.
- Il diagramma seguente mostra un array di interi chiamato c, contenente cinque elementi:



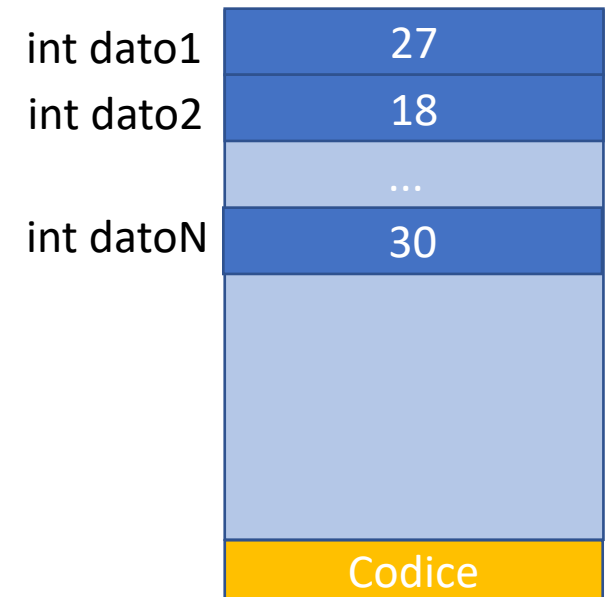
Gli Array

- È spesso necessario memorizzare collezioni di dati omogenei per elaborarle in seguito (statistica, ordinamento, selezione, etc.)
 - I voti di una sessione di esame
 - Le temperature di un periodo
 - L'andamento di un titolo in borsa
- Non ci interessa dare un nome ad ogni elemento
- È sufficiente allocare N elementi ed accedere ad ogni i -esimo elemento della collezione

int dato1	27
int dato2	18
	...
int datoN	30
	Codice

Gli Array

- Gli array sono collezioni di elementi dello stesso tipo allocati in aree contigue di memoria
- Il linguaggio C dispone di sintassi per **allocare** l'array ed **accedere** all'i-esimo elemento
- Per ora ci concentreremo sugli array di dimensione nota al momento della compilazione e allocati nel frame della funzione *main()*

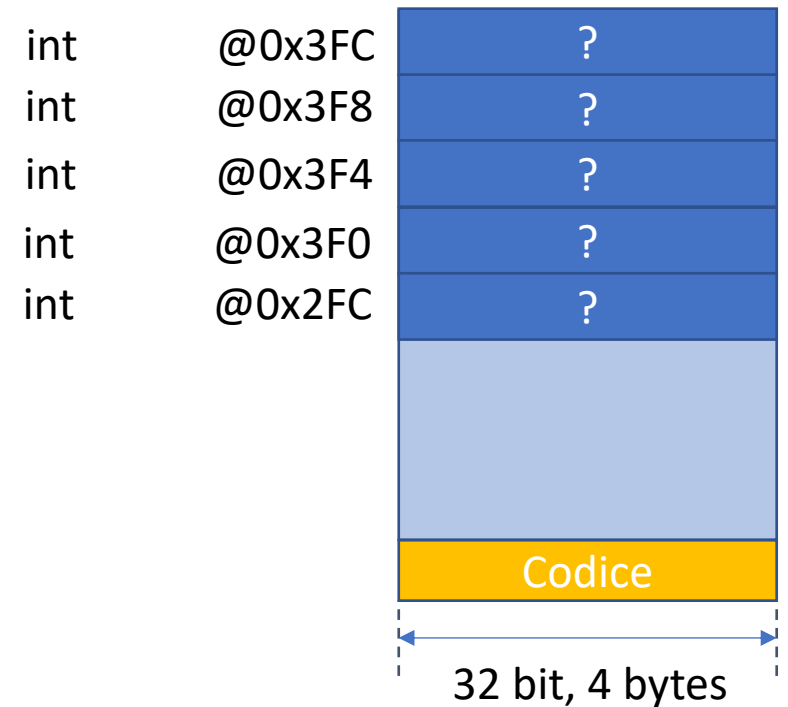


Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Allocazione di un array

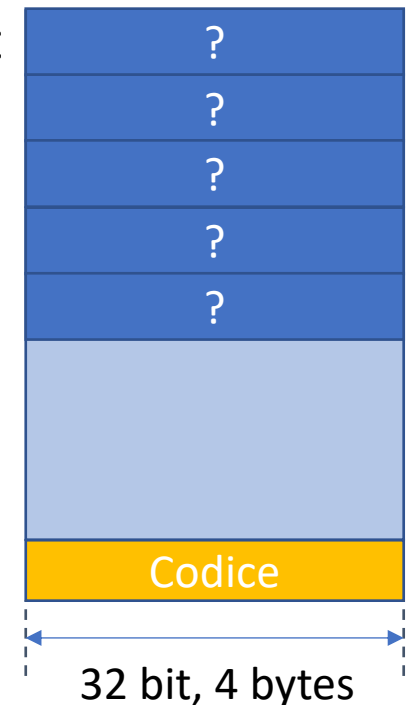
- Un array di N elementi si alloca come
`<tipo> <nome array>[<numero elementi>]`
- Per esempio, per allocare un array di nome voti di 5 int nel frame del main(), scriveremo
`int voti[5] ;`
- Ciò risulta nell'allocazione di 5 parole da 4 bytes l'una nel frame di main() ad indirizzi consecutivi
- Il contenuto di tali parole è indefinito



Allocazione di un array

- L'accesso al valore dell' i -esimo elemento dell'array avviene in lettura e scrittura come
 <nome array>[<indice>]
- **L'indice va da 0 a N-1** (nell'esempio, da 0 a 4)
- L'indice 0 corrisponde sempre all'indirizzo più basso (verso la testa dello stack)

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC



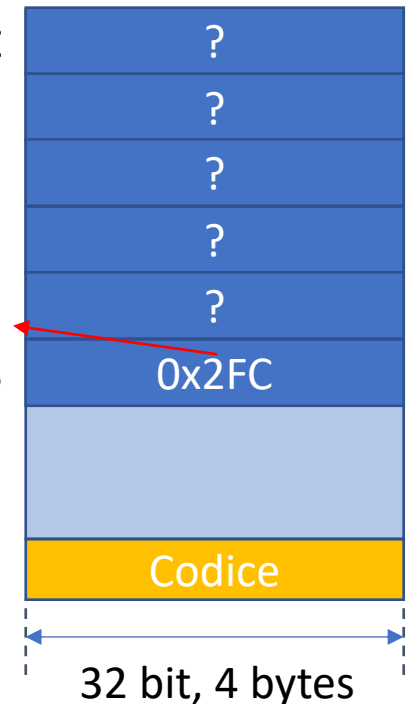
Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Allocazione di un array

- L'accesso al valore dell' i -esimo elemento dell'array avviene in lettura e scrittura come
`<nome array>[<indice>]`
- **L'indice va da 0 a N-1** (nell'esempio, da 0 a 4)
- L'indice 0 corrisponde sempre all'indirizzo più basso (verso la testa dello stack)

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
int* pArr@0x2F8



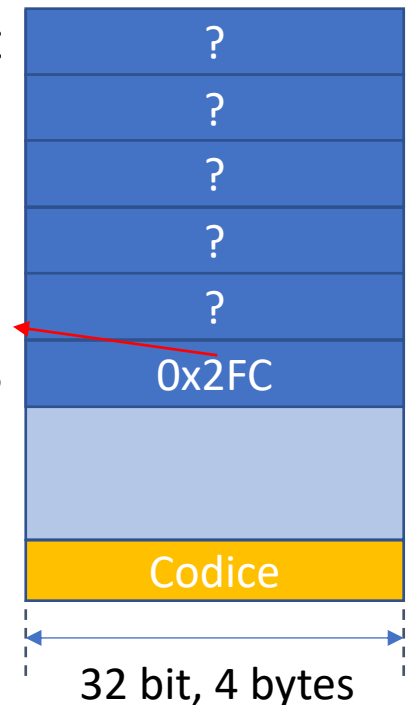
Array e puntatori

- Il nome dell'array è un puntatore che corrisponde all'indirizzo dell'elemento di indice 0

```
int* pArr = voti
```

- L'indirizzo degli elementi successivi è calcolato come l'indirizzo dell'elemento di indice 0 più l'indice moltiplicato per la dimensione del tipo in bytes, perciò gli array sono di tipo omogeneo, contigui in memoria a partire dall'indirizzo più basso e l'indice parte da 0 e non da 1
- Il compilatore C solleva il programmatore dalla gestione della memoria a basso livello con le []

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
int* pArr@0x2F8
```



Outline

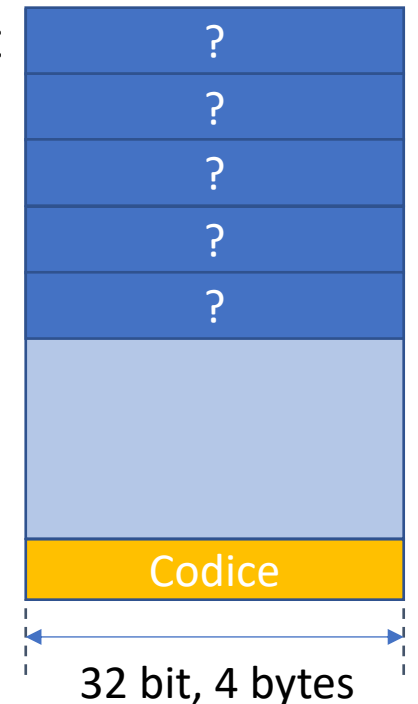
- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Assegnazione di valore

- Assegnazione di un valore all'elemento i -esimo

`<nome array>[<indice>] = <valore>`

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
```



Assegnazione di valore

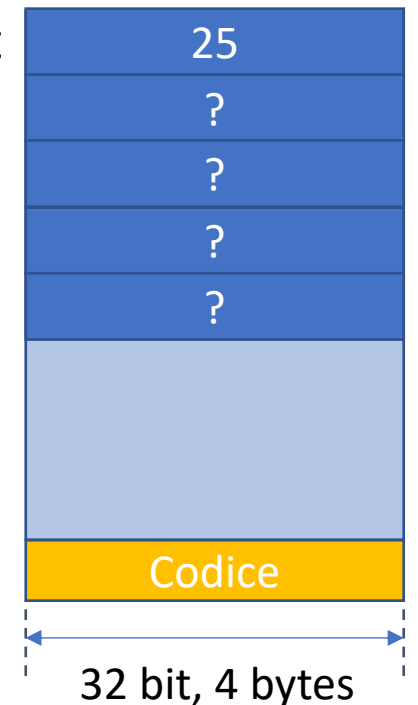
- Assegnazione di un valore all'elemento i -esimo

`<nome array>[<indice>] = <valore>`

per esempio

```
voti[4]=25;
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
```



Assegnazione di valore

- Assegnazione di un valore all'elemento i -esimo

`<nome array>[<indice>] = <valore>`

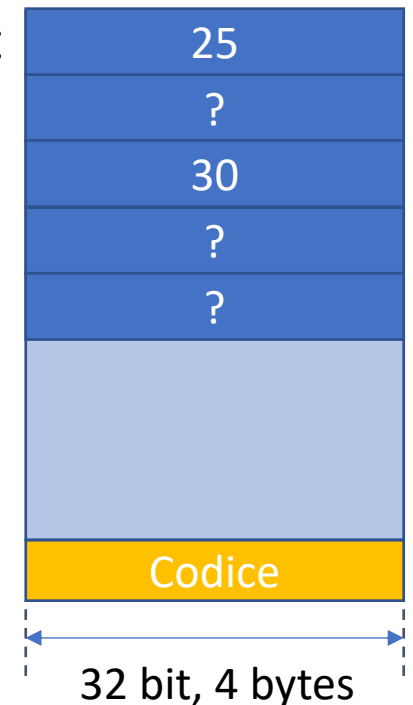
per esempio

```
voti[4]=25;
```

- È possibile accedere all'indirizzo dell'elemento i -esimo tramite referenziazione (altri modi possibili)

```
printf("Inserisci un voto: ");  
scanf("%d ", &voti[2]);
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```



```
$ ./leggi_voto  
Inserisci un voto : 30
```


Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Lettura di valore

- Lettura valore all'elemento i -esimo e assegnamento

`<variabile> = <nome array>[<indice>]`

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```



Lettura di valore

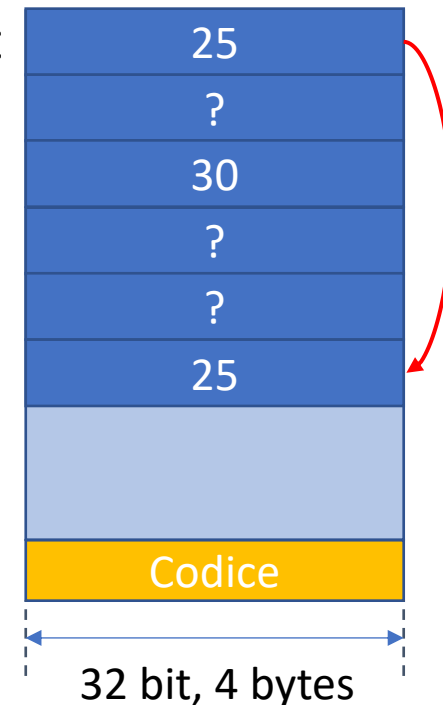
- Lettura valore all'elemento i -esimo e assegnamento

`<variabile> = <nome array>[<indice>]`

- per esempio

```
int ultVoto = voti[4];
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC  
int ultVoto@0x2E8
```



Lettura di valore

- Lettura valore all'elemento i -esimo e assegnamento

`<variabile> = <nome array>[<indice>]`

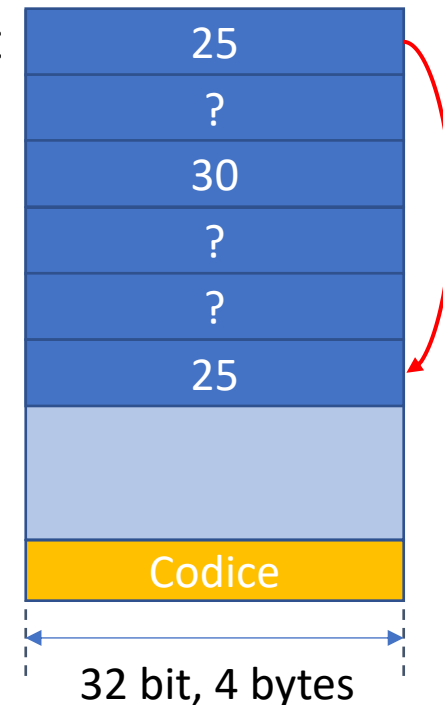
- per esempio

```
int ultVoto = voti[4];
```

- oppure

```
printf("voto: %d", voti[4]);
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
int ultVoto@0x2E8
```



Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Accesso tramite indice

- Spesso si vuole eseguire la stessa operazione (memorizzazione input da tastiera, elaborazione, stampa) su ogni elemento dell'array
- Tipicamente, l'indice dell'elemento non è un valore costante ma una variabile indice che viene fatta variare da 0 a N-1
- Per questa variabile indice utilizzeremo il tipo *int*
 - Il libro utilizza `size_t` , equivalente ad un *unsigned int*
- L'indice viene fatto variare da 0 a N-1
- Definiremo la dimensione dell'array con una `#define` per ora

Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    ➔ int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC



Accesso tramite indice

```
#define SIZE_A 5  
int main(void) {  
    int voti[SIZE_A];
```



```
    for (int i = 0; i < SIZE_A; i++) {  
        voti[i] = i + 18;  
    }
```

```
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, voto[i]);  
    }  
}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

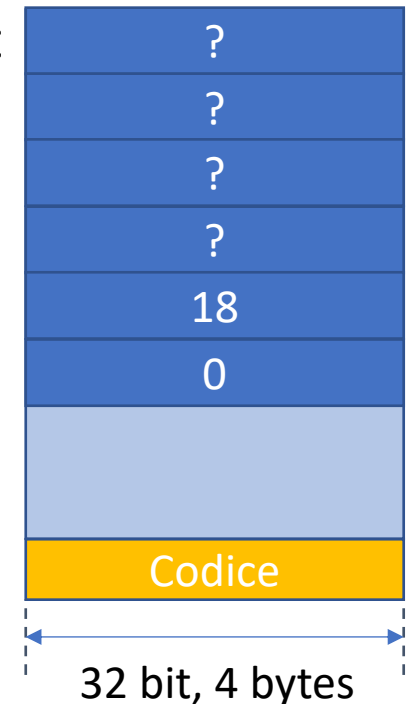
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC

int i



Accesso tramite indice

```
#define SIZE_A 5  
int main(void) {  
    int voti[SIZE_A];
```

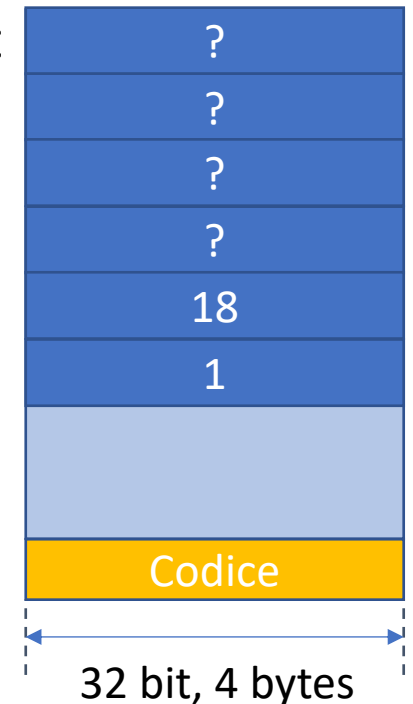
➔

```
    for (int i = 0; i < SIZE_A; i++) {  
        voti[i] = i + 18;  
    }
```

```
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, voto[i]);  
    }  
}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

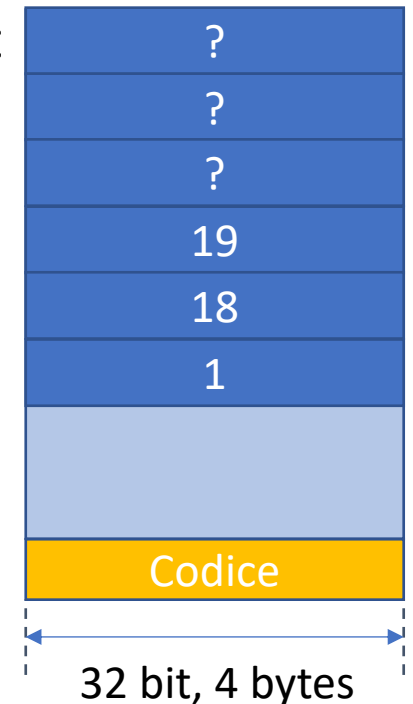
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

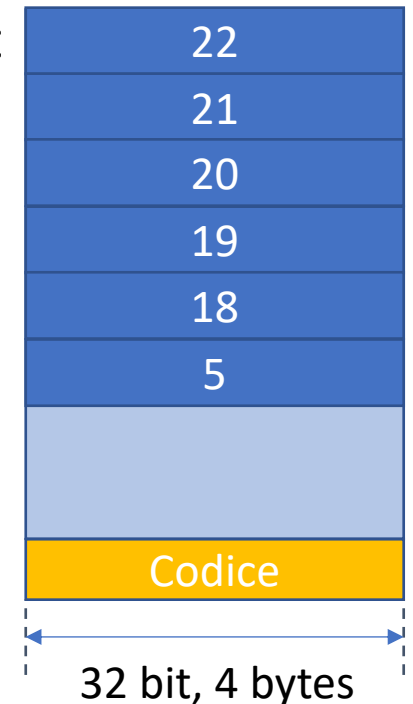
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }
```



```
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

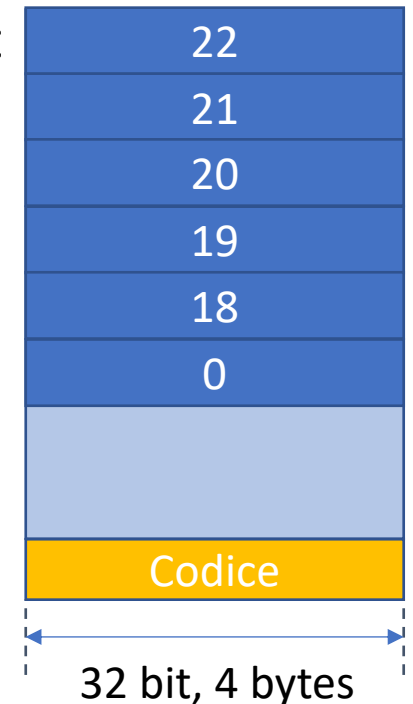
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }
```

➔

```
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
```

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

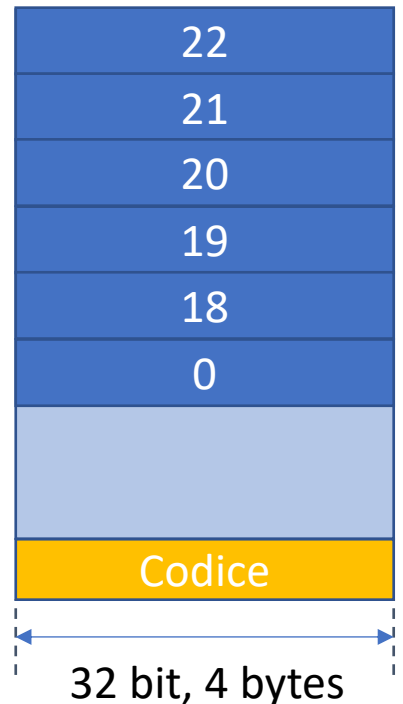
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }
```

→

```
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC
```

int i



```
$ ./fig06_01
0      18
```


Accesso tramite indice

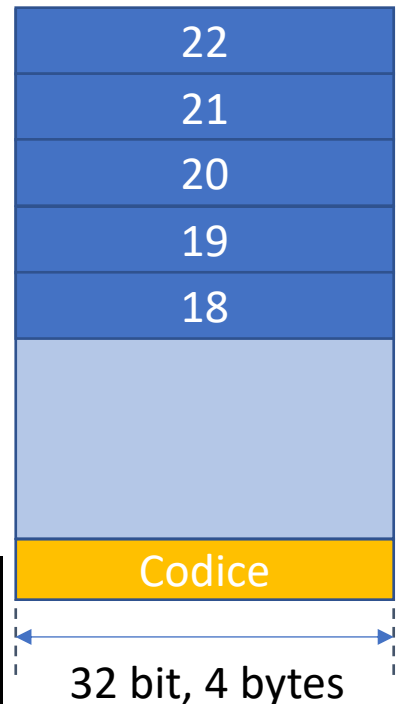
```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x2FC



```
$ ./fig06_01
0      18
1      19
2      20
3      21
4      22
```

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Array predefiniti

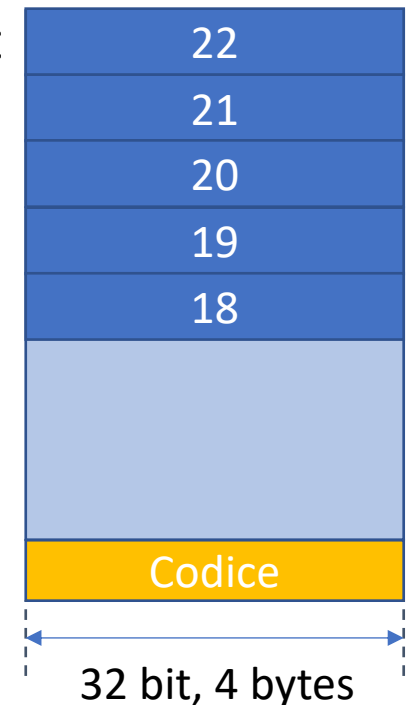
- Il linguaggio C permette di definire array tramite una lista di valori noti

```
int voti[5] = {18, 19, 20, 21, 22}
```

- Equivalente a

```
int voti[] = {18, 19, 20, 21, 22}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```



Array predefiniti

- Il linguaggio C permette di definire array tramite una lista di valori noti

```
int voti[5] = {18, 19, 20, 21, 22}
```

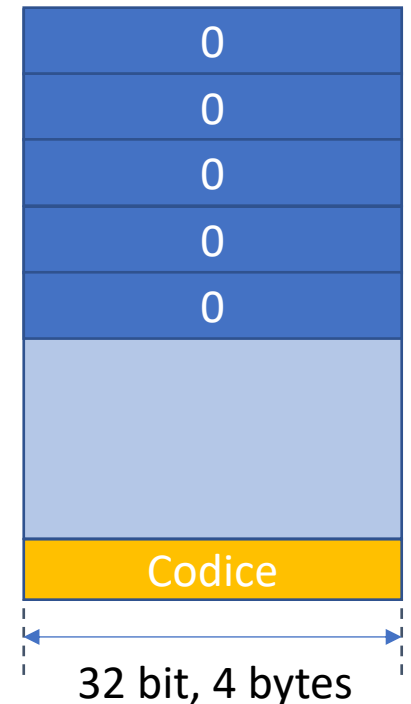
- Equivalente a

```
int voti[] = {18, 19, 20, 21, 22}
```

- Utile per inizializzare un array a zero

```
int voti[5] = {0}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```



Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Buffer overflow

- L'accesso alla cella `SIZE_A`-esima comporta una lettura o scrittura al di fuori dell'area di memoria allocata per l'array
- Nel migliore dei casi, il programma si blocca riportando errore di segmentazione o simili
- Frequentemente, vengono letti dalla memoria dati non corretti che compromettono la correttezza dell'output immediato
- Nel peggiore dei casi, si va a modificare inavvertitamente altre strutture dati senza che il problema si manifesti immediatamente

Buffer overflow

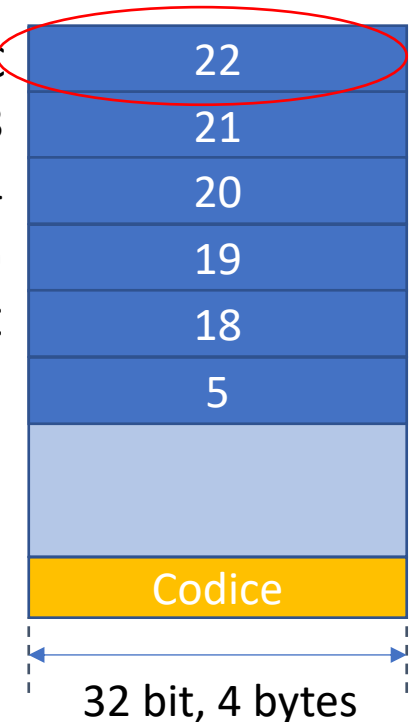


```
#define SIZE_A 5
```

```
int main(void) {  
    int voti[] = {18, 19, 20, 21, 22}  
  
    for (int i = 0; i <= SIZE_A; ++i) {  
        printf("%d\t%d", i, voto[i]);  
    }  
}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x2FC
```

int i



```
$ ./fig06_01  
0      18  
1      19  
2      20  
3      21  
4      22
```